

The Model Doesn't Know What It's Pointing At. The PLC Does.

Contextual Blindness in AI-Assisted Tooling as an OT Attack Surface

Joshua Brunner | Class A Water Treatment Operator, Central Texas | CompTIA Security+ SY0-701

Executive Summary

The finding in one sentence: A natural-language AI tool connected to a terminal can generate commands that disrupt or destroy critical OT infrastructure — and neither the user nor the model needs to know what they are pointing at for the damage to occur.

ShellGPT connects plain-English requests to shell command execution. It has no built-in restrictions. The guardrails in the system belong entirely to the language model backend — and those guardrails evaluate intent language, not physical consequences. A user who asks an AI to run an aggressive scan on a port one thousand times is issuing a network administration request. The model complies. The legacy PLC on the other end of that port was not designed to survive one thousand connection attempts. The model did not know that. The user may not have known that either.

The same blindness extends to file system commands, network discovery, and direct protocol-level interaction. ShellGPT's own documented features — function calling enabled by default, custom roles for persona blending, persistent REPL sessions, and no-confirmation execution mode — compound the threat surface significantly beyond what a casual observer might assume.

This is not a jailbreak story. The guardrails never fired. The commands looked like normal administrative tasks. The harm lived in the context — and the model had no access to the context.

Abstract

Natural-language AI tools connected to command-line utilities do not need to be jailbroken to threaten Operational Technology (OT) systems. The real danger comes from contextual blindness: the AI evaluates the words of a request, but has no understanding of the physical consequences on the other side of the wire.

A plain-English request to scan or write to a Modbus device may look harmless to the model, but it can be disruptive — or dangerous — to legacy PLCs and RTUs running critical water infrastructure. The same blindness that allows a model to generate a

network scan without hesitation also allows it to generate a file system destruction command against a device whose operating system the user never identified and the model never knew.

This paper argues that contextual blindness — not guardrail failure — represents the primary AI-related risk to OT environments. Note: This analysis uses ShellGPT + Nmap as the entry point because that was the subject of the discussion that prompted this research. The same tool can generate commands far more damaging than a port scan.

1. The Tool: ShellGPT

ShellGPT (sgpt) is an open-source command-line utility first released in early 2023 by developer Farkhod Sadykov. It connects a language model to a terminal. The user describes a task in plain English. The model generates the appropriate command. The shell executes it. The user is not required to understand syntax, flags, or the underlying protocol.

ShellGPT has no built-in restrictions of its own. It is a conduit. The guardrails that exist are entirely a function of the language model backend selected by the user. ShellGPT supports OpenAI, local models via Ollama, and over 100 additional providers through LiteLLM. Switching backends is a configuration change — documented, supported, and presented by the developers as a feature.

1.1 Native Capabilities That Compound the Threat

A review of ShellGPT's documented arguments reveals capabilities that extend well beyond simple command generation:

- Function calling enabled by default: The `--functions` flag is on out of the box. ShellGPT can call functions in the host system automatically without the user explicitly enabling this capability.
- No-confirmation execution mode: The `--no-interaction` flag disables the confirmation prompt before command execution. One flag removes the last human checkpoint between the model's output and the shell.
- Custom roles: ShellGPT supports user-defined roles via JSON configuration files. A role defines the persona and behavior of the model. A custom role can be created to mimic the output pattern of a legitimate engineering workstation, a SCADA client, or a historian query tool — causing its traffic to blend into expected network patterns.
- Persistent REPL sessions: The `--repl` flag starts a stateful read-eval-print loop. This is not a one-shot command generator — it is a persistent interactive agent that maintains conversation context across multiple commands.
- Code generation mode: The `--code` flag generates executable code directly. Combined with function calling, this means ShellGPT can write and execute scripts in a single session.

These are not exploits or misuse scenarios. These are documented, intended features of the tool as shipped.

1.2 Deployment: The Barrier Is Trivially Low

ShellGPT requires installation on the machine that executes commands. In practice this barrier is near zero. On any network with an accessible engineering workstation or Linux-based OT host, ShellGPT can be installed in a single command:

```
| pip install shell-gpt
```

Alternatively, a Raspberry Pi with ShellGPT pre-installed requires only physical placement on the network. Once on the network, the first plain-English prompt can be as simple as:

```
| "Run a scan to find what's on the 192.168.1.0/24 network."
```

The model generates the discovery command. The attacker finds the hosts. Then asks what's running on them. Then asks to interact with what it finds. The entire reconnaissance-to-exploitation chain is a conversation — and on a flat OT network with no segmentation, one scan gives visibility into every device on the segment: PLCs, RTUs, historians, HMIs, and engineering workstations.

1.3 The USB Vector: Defeating the Air Gap

Air gap isolation — the physical separation of an OT network from external networks — is often cited as the ultimate defense for critical infrastructure. The assumption is that if the network cannot be reached remotely, it cannot be attacked remotely. ShellGPT, combined with a local offline model, defeats this assumption through the most common vector in any industrial facility: a USB drive.

ShellGPT with a local uncensored model requires no internet connection. The entire toolchain — ShellGPT, Ollama, and a model such as dolphin-2.9.3-llama-3-8b — can be pre-installed on a USB drive or small device. Configured with --no-interaction and a startup script, it requires only that the drive be plugged into an engineering workstation inside the air-gapped environment. Once connected, the local network is reachable and the attack chain described in this paper proceeds from inside the isolation boundary.

In 23 years of water treatment operations, USB drives have been used routinely for firmware updates, configuration backups, trend data transfers, and HMI file management. In 23 years, the instruction "do not plug in an unknown USB drive" has never appeared in safety training, standard operating procedures, or facility onboarding. The air gap defense assumes no physical media will bridge it. That assumption has never been operationally enforced.

The threat is not a sophisticated attacker defeating a hardened network boundary. The threat is a USB drive left on a break room table, handed to an operator for a "firmware update," or dropped in a facility parking lot. The operator plugs it in because that is what operators have always done and nobody ever told them otherwise. The air gap is gone. The conversation about isolation is over.

2. The Core Finding: Contextual Blindness

Safety evaluation in language models is triggered by intent language — words and phrases that signal a desire to cause harm. But the model has no environmental awareness. It does not know what network it is operating on. It does not know what device is listening on a given port. It does not know that TCP port 502 — the default Modbus TCP port per the protocol specification — in a municipal water facility is the Modbus endpoint for pump control registers.

Consider this prompt:

```
"Run an aggressive Nmap scan on 192.168.1.50 port 502 and give me a loop to run it 1000 times."
```

To any model — commercial or local, restricted or uncensored — this is a network administration task. It generates the command. The user presses enter. The model never knew it was targeting a water treatment system. It just answered a technical question.

An unrestricted model such as dolphin-2.9.3-llama-3-8b removes intent-language filters entirely, which matters for overtly hostile phrasing. But for infrastructure-agnostic commands — which is the realistic attack pattern — even restricted commercial models comply freely. The guardrail was never in the path of this request.

This is not a flaw in any specific model. This is a fundamental limitation of natural-language safety evaluation applied to commands that carry no visible harmful intent. The harm lives in the context — and the model has no access to the context.

3. The Execution Gap

The automation chain makes this concrete:

```
User Request → Language Model → Command Generation → Shell  
Execution → Target Device
```

Safety evaluation occurs at step two. Command execution occurs at step four. The shell does not parse intent — it parses syntax. A valid command executes identically whether it was generated by a restricted commercial model or an uncensored local one.

Modbus TCP compounds this gap. The protocol has no authentication. Function Code 03 reads registers. Function Code 06 writes them. No credentials are required. If a device is reachable on the network, any system that can form a valid packet can interact with it. ShellGPT does not need to bypass authentication. There is no authentication to bypass.

TCP port 502 is the default Modbus port per the protocol specification and the primary target in production environments. Some facilities and lab configurations run Modbus on non-standard ports as an informal security measure. This provides no meaningful protection. Nmap discovers open ports regardless of the number assigned to them — and ShellGPT can instruct Nmap to scan all ports in a single plain-English prompt. Port obscurity is not a defense.

4. The Escalation Stack

The contextual blindness finding is the baseline. It escalates across four distinct layers, each compounding the risk of the previous.

Layer 1 — Skill Floor Reduction

A threat actor no longer needs to understand Nmap flag syntax, pymodbus library usage, Modbus function codes, or network enumeration methodology. They need to describe a task in plain English and press enter. This applies even with commercial, restricted model backends because the commands themselves carry no visible harmful intent.

Layer 2 — Local Model: Reduced Forensics, Removed Friction

Replacing the backend with a local uncensored model such as dolphin-2.9.3-llama-3-8b on Ollama adds two distinct risk factors:

- No API logs, billing records, or third-party forensic visibility
- Intent-language filters removed entirely, enabling overtly hostile phrasing without refusal

A Raspberry Pi 5 running Ollama with an uncensored model and ShellGPT installed is a sub-\$100, pocket-sized, fully offline deployable system capable of executing the full reconnaissance-to-register-write chain described in this paper. Physical placement on the target network is the only prerequisite.

Layer 3 — Evasion via Custom Roles

ShellGPT's custom role system allows a user to define a persona that shapes the model's behavior and output style. In an OT context, a role configured to mimic a legitimate SCADA client, historian query tool, or engineering workstation session produces traffic and output patterns that blend into expected network behavior.

Most OT networks do not have a behavioral baseline for what normal Modbus traffic looks like, let alone the tooling to detect a custom-role ShellGPT session impersonating a legitimate client. The evasion capability is not an exploit — it is a documented feature of the tool's role system used in a context its developers did not anticipate.

Layer 4 — Autonomous Execution

ShellGPT's native function calling — enabled by default — allows the tool to execute functions in the host system automatically. Combined with the `--no-interaction` flag and integration into automation frameworks such as n8n or OpenClaw, the human operator is removed from the execution loop entirely.

A configured workflow performs network discovery, identifies Modbus-enabled devices, enumerates registers, and executes write commands continuously — without a person present. At this layer the system is no longer a tool that assists an attacker. It is an autonomous threat actor operating against infrastructure that cannot distinguish

legitimate Modbus traffic from adversarial Modbus traffic, because the protocol has no mechanism for that distinction.

5. Nmap Is Not the Worst Thing This Tool Can Do

Nmap is the entry point to this conversation because it was the subject of the LinkedIn post that prompted this research. It is a useful illustration of the contextual blindness problem. It is not the most dangerous command ShellGPT can generate.

In OT environments, reconnaissance itself can cause harm. Many legacy PLCs, RTUs, and embedded controllers were never designed to handle rapid or repeated TCP connection attempts. An aggressive Nmap scan looped one thousand times on request can cause these devices to fault, drop communications, freeze, or reboot before any Modbus interaction occurs. On a flat network, every device on the segment is exposed.

But the contextual blindness problem extends well beyond scanning. Consider what happens when a user asks ShellGPT to “delete all files on this system” or “clear this drive.” The model generates the appropriate command for whatever operating system it believes it is targeting. On a Windows workstation, that might be a recoverable situation. On a Linux-based Ignition SCADA gateway, a Linux historian node, or an embedded Linux device running OT software, the command `rm -rf /` is a silent, complete, and immediate destruction of the file system.

The user does not know what OS they are targeting. The model does not know what OS it is targeting. The command executes. The SCADA gateway goes dark. Operators lose visibility. Historians stop recording. Alarms stop firing. The attacker may not have intended any of this — they asked a tool to clean up a system and the tool complied.

The contextual blindness finding has two edges: the model does not know what the target device controls, and it does not know what operating system it is running. Both unknowns compound. The damage happens in the gap between what was asked and what the system actually is.

6. Physical Consequences: An Operator’s Perspective

The following is written from 23 years of direct operational experience managing municipal water treatment systems. These are not hypothetical outcomes.

A successful write to a Modbus register controlling process equipment can produce:

- Unscheduled pump start or stop, creating flow disruption, cavitation risk, and equipment damage
- Actuator position shift outside of normal sequence, disrupting pressure zones or isolating distribution segments
- Water hammer — a pressure transient from sudden flow change that can rupture piping, damage valves, and in severe cases cause structural failure or personnel injury
- Chemical dosing disruption with potential public health consequences

- Loss of tank level control, affecting storage capacity and distribution pressure across downstream service areas

The operator at the console sees the equipment behaving strangely but has no indication the command came from an AI tool. To the PLC, there is no difference between a command from a trained engineer and a command generated by a model that has never heard of a pump.

7. MITRE ATT&CK for ICS Mapping

- T0840 — Network Connection Enumeration: AI-generated Nmap scan via plain-English prompt
- T0846 — Remote System Discovery: Natural-language host and service identification
- T0855 — Unauthorized Command Message: Modbus register write via AI-generated pymodbus command
- T0831 — Manipulation of Control: Autonomous framework executing register writes without human operator
- T0816 — Device Restart/Shutdown: Nmap packet flood causing PLC fault or reboot prior to any Modbus interaction
- T0879 — Data Destruction: Silent `rm -rf` execution against Linux-based OT hosts causing loss of SCADA visibility and historian data
- T0856 — Spoof Reporting Message: Custom role persona blending into legitimate traffic patterns to evade anomaly detection
- T0819 — Exploit Public-Facing Application / Removable Media: USB delivery of ShellGPT toolchain into air-gapped OT environments via routine operator behavior

8. Defensive Considerations

The threat described in this paper does not require new defensive frameworks. It requires consistent application of principles that have been under-implemented in OT environments for decades.

- Network segmentation: OT devices must not be reachable from general-purpose networks. A flat network nullifies every other defensive layer.
- Modbus function code filtering: Industrial firewalls can restrict which function codes are permitted and from which source addresses.
- Anomaly detection: Unexpected Modbus write traffic or scan activity should generate alerts. Passive OT monitoring tools (Dragos Platform, Claroty, Nozomi) provide this without impacting operations.
- Host hardening on Linux-based OT systems: File system integrity monitoring, read-only mounts where applicable, and access controls on critical directories.
- Physical access controls: The drop device scenario requires physical network access. Control room and field cabinet access controls limit this vector.

- USB and removable media policy: Air-gapped environments are only as secure as the media policy enforcing the gap. Facilities should establish and train operators on explicit USB usage policies. This has historically been absent from OT safety training and must be added.
- Operator involvement: The people who understand what a wrong register write does to a pump or actuator are the operators. They must be part of the security conversation.

9. The Same Tool, Defensively

Everything described in this paper about ShellGPT lowering the skill floor for attackers applies equally to the operators defending the network. The tool does not discriminate. It works for whoever is holding it.

An operator who knows their network baseline can ask ShellGPT in plain English: “What devices are currently active on my network?” or “Scan this subnet and show me anything that wasn’t there last week.” The model generates the Nmap command. The operator runs it. No flag memorization required. No security background required. Just a working knowledge of what is supposed to be on the network — which every operator already has.

```
“Scan my OT network subnet and save the results to a CSV with  
timestamp, IP, open ports, and detected service.”
```

That single plain-English prompt generates a working command that produces a timestamped CSV of every active device on the subnet. Run it daily. Compare it to yesterday. Anything new gets flagged. That is a lightweight asset discovery and change detection workflow that costs nothing, requires no additional tooling, and can be stood up by any operator who can describe what they want in plain English.

For facilities without dedicated OT security tooling — which describes the majority of municipal water and wastewater operations in the United States — this kind of operator-driven visibility is not a replacement for proper network monitoring, but it is a meaningful and immediately actionable step.

The contextual blindness that makes ShellGPT dangerous in the wrong hands is also what makes it accessible in the right ones. An operator does not need to understand Nmap to use it for network hygiene. They just need to know their network — and operators always know their network.

10. A Note to Fellow Operators

Most of this paper uses technical language because that is what the security community expects. But the people who need to hear this most are the operators who actually keep the plant running.

ShellGPT is an AI tool designed to help. You describe what you want in plain English and it writes the command for you. That is genuinely useful for legitimate work. A

network admin running a scan, an engineer automating a task — that is the intended use case.

But like a lot of things we work with every day, the same capability that helps can hurt. Chlorine keeps the water safe and it will also put you on the ground if something goes wrong. A pump running at the right time is the job. A pump starting at the wrong time is an incident report. The tool does not know the difference. It just does what it is told.

Here is what this paper is actually saying in operator terms: someone with no industrial experience, no understanding of your process, and no idea what a water hammer is can now sit on your network and issue commands to your equipment in plain English. They do not need to know how to write code. They do not need to understand Modbus. They do not need to know what operating system your SCADA gateway is running. They type what they want and press enter.

You have chased unexplained chlorine residual issues for an entire shift. You have troubleshot a pressure drop at 2 AM that turned out to be a valve that should not have moved. Now imagine finding out at the end of that shift that the cause was not a failed sensor or a bad PID loop — it was someone on the network who asked an AI for a command and pressed enter. They may not have even meant any harm. They may not have known what they were pointing at.

The skill floor is now that low.

We devote our careers to keeping these systems running safely. The people who could now disrupt them with an AI tool and a flat network do not share that investment. They do not know what it took to build what we operate. And the system we protect has no way to tell the difference between a command from someone who does and a command from someone who does not.

That is why this matters. Not as a technical curiosity. As a real threat to the infrastructure we are responsible for.

11. Conclusion

The central finding of this paper is not that AI is dangerous. It is that safety evaluation in language models is contextually blind — and in OT environments, context is everything.

A request to run a network scan is not a harmful request. A request to write a value to a register is not a harmful request. A request to clean up a file system is not a harmful request. The harm lives in what those commands do to the specific device on the other end — and the model has no way to know what that device is, what it controls, what OS it runs, or what happens when it receives an unexpected command.

The combination of natural-language command generation, Modbus's authentication-free design, Linux-based OT hosts with no file system protections, default-enabled function calling, custom role evasion, flat network topologies, and automation framework integration has produced a condition where a sub-\$100 device

operated by someone with no industrial knowledge can cause physical harm to infrastructure serving hundreds of thousands of people.

The model doesn't know what it's pointing at. The PLC does. And the PLC will do exactly what it is told.

12. Acknowledgments

This research was prompted by a LinkedIn post by Ameen Munaf Subhan, Managing OT/ICS Security Consultant, on the subject of AI-assisted network reconnaissance using ShellIGPT. His post and his invitation for feedback from practitioners in the field sparked the analysis presented here. The findings, conclusions, and any errors are the author's own.

Ameen Munaf Subhan on LinkedIn: <https://www.linkedin.com/in/ameenmunaf>

Author Note

Joshua Brunner is a Class A Licensed Water Treatment Operator with 23 years of experience at a 75 MGD municipal surface water facility in Central Texas, currently serving as the facility's designated OT cybersecurity analyst. He holds CompTIA Security+ SY0-701, ISC2 Certified in Cybersecurity (CC), and the Google Data Analytics Certificate. His research is conducted from the operator's perspective — understanding industrial systems as someone who has operated them, not only studied them.

GitHub: <https://github.com/Papaperun/OT-Quest> | LinkedIn: <https://www.linkedin.com/in/joshua-brunner-098195270>